

Manhattan Travel App

登录 / 注销功能 —— iOS SwiftUI 代码详解

配套 Swift / SwiftUI 知识点整理（含 Swift Book 章节标注）

项目路径：ios/ManhattanTravelApp | 语言：Swift（SwiftUI） | 平台：iOS

文档说明

本文档逐一讲解「登录 / 注销」功能涉及的全部 Swift

源文件：每个文件先给出完整代码，再分段解释代码逻辑，并在结尾用「知识点标签」标出涉及的 Swift / SwiftUI 概念，同时标注对应的《The Swift Programming Language》官方文档章节（● Swift Book 参考章节，可点击）。文档末尾附有按主题分类的知识点速查表，方便复习。

本功能目前为「前端 Mock 实现」：不连接后端，`AuthManager.login(...)` 只在本地校验邮箱格式与密码长度（ ≥ 6 位），校验通过即视为登录成功，并用 `UserDefaults` 记住登录状态；`logout()` 清空状态。后续若需接入项目自带的 FastAPI 后端（`backend/app/routers/auth.py` 中的 `/auth/login`、`/auth/signup`），只需替换 `AuthManager` 内部实现，上层视图（`LoginView`、`RegisterView`、`ProfileView`）几乎不需要改动。

目录

1. ManhattanTravelAppApp.swift —— App 启动入口
2. ContentView.swift —— 根视图（登录态路由）
3. Models/User.swift —— 用户数据模型
4. Managers/AuthManager.swift —— 登录/注销核心逻辑 ★
5. Views/LoginView.swift —— 登录界面
6. Views/RegisterView.swift —— 注册页（占位骨架）
7. Views/MainTabView.swift —— 主标签栏（登录后的主界面）
8. Views/ProfileView.swift —— 个人主页（含登出功能）★
9. 项目结构总览
10. Swift / SwiftUI 知识点速查表

1. ManhattanTravelAppApp.swift — App 启动入口

■■■■■ManhattanTravelApp/ManhattanTravelAppApp.swift

整个 App 的启动点。创建全局唯一的 `AuthManager`（登录状态管理器），并通过环境对象注入到所有视图中，是后续登录/注销状态能在各个页面间共享的根本原因。

完整代码

```
import SwiftUI

@main
struct ManhattanTravelAppApp: App {
    @StateObject private var authManager = AuthManager()

    var body: some Scene {
        WindowGroup {
            ContentView()
                .environmentObject(authManager)
        }
    }
}
```

代码讲解

- 第 3 行 `@main`：告诉编译器「程序从这里开始」。整个 App 只能有一个 `@main`。
- 第 4 行 `struct ManhattanTravelAppApp: App`：遵循 App 协议的结构体即代表一个完整应用；App 协议只要求实现一个 `body: some Scene` 计算属性。
- 第 5 行 `@StateObject private var authManager = AuthManager()`：在 App 生命周期内创建并持有唯一一个 `AuthManager` 实例。`@StateObject` 保证它只会被创建一次，不会因视图刷新而重建——是整个登录状态的「单一数据源 (Single Source of Truth)」。`@StateObject` 属于「属性包装器 (Property Wrapper)」，在 Swift Book 属性一章有详细讲解。
- 第 8 行 `WindowGroup`：最常用的 Scene，代表应用的一个窗口（iPhone 上通常只有一个）。
- 第 10 行 `.environmentObject(authManager)`：把 `authManager` 放进 SwiftUI 的「环境 (Environment)」。`ContentView` 及其所有子视图（`LoginView`、`ProfileView` 等）都可以用 `@EnvironmentObject` 直接取到同一个实例，无需手动一层层传参——这就是「依赖注入」。

知识点：[@main / App 协议 / Scene 与 WindowGroup / @StateObject / 环境对象注入 .environmentObject\(_:\)](#)

● Swift Book 参考章节：[《属性》](#) / [《协议》](#)

2. ContentView.swift —— 根视图（登录态路由）

■■■■■ManhattanTravelApp/ContentView.swift

整个 App 的「路由器」：根据 `authManager.isLoggedIn` 决定显示登录页还是登录后的主界面。这正是登录成功 / 注销后界面自动切换的核心逻辑。

完整代码

```
import SwiftUI

struct ContentView: View {
    @EnvironmentObject var authManager: AuthManager

    var body: some View {
        Group {
            if authManager.isLoggedIn {
                MainTabView()
            } else {
                LoginView()
            }
        }
        .animation(.default, value: authManager.isLoggedIn)
    }
}

#Preview {
    ContentView()
    .environmentObject(AuthManager())
}
```

代码讲解

- 第 4 行 `@EnvironmentObject var authManager: AuthManager`：从环境中取出 App 里注入的同一个 `authManager` 实例，本视图不需要自己创建它。
- 第 7~13 行 `Group { if ... else ... }`：if/else 写在 SwiftUI 的 `@ViewBuilder` 上下文里会被编译成「条件视图」；`Group` 把两种可能性打包成一个整体，方便统一添加修饰符。`@ViewBuilder` 本质上是「结果构建器 (Result Builder)」，允许用「声明式」方式把多个视图表达式组合成一个闭包返回值，这正是闭包在 SwiftUI 中的核心应用场景（Swift Book 闭包章节）。
- 第 8 行 `authManager.isLoggedIn`：直接读取被 `@Published` 标记的属性。因为本视图通过 `@EnvironmentObject` 订阅了 `authManager`，这个值一旦改变，`body` 就会自动重新求值并刷新界面——登录成功后 `isLoggedIn` 变 `true`，这里就会从 `LoginView` 切换到 `MainTabView`；点击「Log Out」后变回 `false`，又会自动切回 `LoginView`。
- 第 14 行 `.animation(.default, value: authManager.isLoggedIn)`：当 `isLoggedIn` 的值变化时，为这次视图切换自动加上默认过渡动画（淡入淡出）。
- 第 18~21 行 `#Preview { ... }`：Swift 5.9 引入的「预览宏」，取代旧的 `PreviewProvider`，让 Xcode Canvas 实时渲染该视图；预览环境里没有 App 的环境对象，所以要手动 `.environmentObject(AuthManager())` 提供一份。

知识点：`@EnvironmentObject / Group / @ViewBuilder` 中的条件视图（结果构建器 + 闭包） / `.animation(_:value:)` / `#Preview` 宏

● Swift Book 参考章节：《闭包》 / 《基础语法》 / 《属性》

3. Models/User.swift — 用户数据模型

■■■■■ManhattanTravelApp/Models/User.swift

定义「用户」这一份纯数据模型 (Model)，字段与 Profile 截图里展示的内容一一对应：姓名、邮箱、加入时间、Pro 标识、行程数、地点数、错峰出行占比。

完整代码

```
import Foundation

struct User {
    var name: String
    var email: String
    var joinedDate: String
    var isPro: Bool
    var tripsCount: Int
    var placesCount: Int
    var offPeakPercentage: Int
}
```

代码讲解

- `import Foundation`：引入基础库（这里主要是为后续扩展预留，例如未来加入 `Date`、`UUID` 等类型）。
- `struct User`：使用 `struct`（值类型）而不是 `class`。值类型在赋值/传递时会被整体拷贝，更安全、可预测，非常适合表示「某一时刻的数据快照」，这也是 Apple 推荐的 Model 设计方式。（Swift Book → 结构体与类章节详细对比了两者的区别）
- 7 个 `var` 存储属性：`name`、`email`、`joinedDate`、`isPro`、`tripsCount`、`placesCount`、`offPeakPercentage`。分别对应截图中的「Amelia Chen」「Joined March 2026」「Pro」徽章、「3 Trips」「27 Places」「94% Off-peak」。（Swift Book → 属性章节：存储属性 vs 计算属性）
- 这里没有写任何 `init`，Swift 会自动为 `struct` 生成一个「逐成员初始化器 (memberwise initializer)」，即可以写 `User(name: ..., email: ..., joinedDate: ..., ...)`。`AuthManager.mockUser` 正是用这种方式构造出来的。（Swift Book → 初始化章节：逐成员初始化器）

知识点：`struct`（值类型 vs 引用类型）/ 存储属性 (stored property) / 逐成员初始化器 (memberwise initializer)

● Swift Book 参考章节：《结构体与类》/ 《属性》/ 《初始化》

4. Managers/AuthManager.swift — 登录/注销核心逻辑 ★

■■■■■ManhattanTravelApp/Managers/AuthManager.swift

整个登录/注销功能的「大脑」。用 mock（本地模拟）数据实现登录态，没有任何网络请求；通过 `ObservableObject` + `@Published` 让所有界面在状态变化时自动刷新，并用 `UserDefaults` 记住登录状态。

完整代码

```
import Foundation

/// Mock authentication state for the mobile UI mockup.
/// No network calls are made — `login` accepts any well-formed credentials
/// and `logout` simply clears the session. Swap this out for a real
/// backend-backed implementation once the auth API is available.
@MainActor
final class AuthManager: ObservableObject {
    @Published var isLoggedIn: Bool
    @Published var currentUser: User?
    @Published var errorMessage: String?

    private let isLoggedInKey = "auth.isLoggedIn"

    static let mockUser = User(
        name: "Amelia Chen",
        email: "amelia.chen@example.com",
        joinedDate: "Joined March 2026",
        isPro: true,
        tripsCount: 3,
        placesCount: 27,
        offPeakPercentage: 94
    )

    init() {
        let storedLoginState = UserDefaults.standard.bool(forKey: isLoggedInKey)
        self.isLoggedIn = storedLoginState
        self.currentUser = storedLoginState ? AuthManager.mockUser : nil
    }

    func login(email: String, password: String) {
        let trimmedEmail = email.trimmingCharacters(in: .whitespacesAndNewlines)

        guard !trimmedEmail.isEmpty, !password.isEmpty else {
            errorMessage = "Please enter your email and password."
            return
        }

        guard trimmedEmail.contains("@") else {
            errorMessage = "Please enter a valid email address."
            return
        }

        guard password.count >= 6 else {
            errorMessage = "Password must be at least 6 characters."
            return
        }

        errorMessage = nil
        currentUser = AuthManager.mockUser
        isLoggedIn = true
        UserDefaults.standard.set(true, forKey: isLoggedInKey)
    }

    func logout() {
        currentUser = nil
        isLoggedIn = false
        errorMessage = nil
        UserDefaults.standard.set(false, forKey: isLoggedInKey)
    }
}
```

代码讲解

- 第 7 行 `@MainActor`：Swift 并发模型中的「全局执行者 (Global Actor)」标注。被标注的类型，其所有属性访问与方法调用都被限定在主线程（UI 线程）上执行——避免在后台线程修改 `@Published` 属性导致界面更新错误或崩溃。（Swift Book → 并发章节：Actor 与数据隔离）
- 第 8 行 `final class AuthManager: ObservableObject`：① `class` 是引用类型——App、ContentView、LoginView、ProfileView 拿到的是「同一个对象」，任意一处修改，其它地方立刻能看到；② `final` 表示禁止被继承（Swift Book → 结构体与类章节）；③ `ObservableObject` 是一个「标记协议 (marker protocol)」，配合 `@Published` 属性会自动合成 `objectWillChange` 这个 Combine 发布者 (Publisher)。（Swift Book → 协议章节）
- 第 9~11 行 `@Published var isLoggedIn / currentUser / errorMessage`：`@Published` 是一个「属性包装器 (Property Wrapper)」——这些属性的值一旦改变，就会自动通知所有订阅了本对象的视图重新计算 body。`User?` 与 `String?` 中的 `?` 表示「可选类型 (Optional)」，代表「可能没有值」：未登录时 `currentUser` 为 `nil`；没有错误时 `errorMessage` 为 `nil`。（Swift Book → 基础语法章节：Optionals；属性章节：属性包装器）
- 第 13 行 `private let isLoggedInKey = "auth.isLoggedIn" : private` 限定只能在本类型内部访问，把 `UserDefaults` 的 key 封装起来，避免外部直接操作。（Swift Book → 访问控制章节）
- 第 15~23 行 `static let mockUser = User(...)`：`static` 表示这是「类型属性」，属于 `AuthManager` 这个类型本身，而不属于某一个实例，所以可以直接写 `AuthManager.mockUser`，所有实例共享同一份数据。这里构造出了截图中「Amelia Chen」的全部资料。（Swift Book → 属性章节：类型属性）
- 第 25~29 行 `init()`：自定义构造器。用 `UserDefaults.standard.bool(forKey:)` 读取上一次保存的登录状态，实现「记住登录」——重启 App 后仍保持登录；再用三元运算符 `storedLoginState ? AuthManager.mockUser : nil` 决定 `currentUser` 的初始值。（Swift Book → 初始化章节；基本运算符章节：三元条件运算符）
- 第 31~53 行 `func login(email:password:)`：① `email.trimmingCharacters(in: .whitespacesAndNewlines)` 去掉首尾空白（Swift Book → 字符串与字符章节）；② 三个 `guard ... else { ...; return }` 依次校验「邮箱和密码不能为空」「邮箱要包含 @」「密码长度 ≥ 6」——这是「提前返回 (early exit)」模式，避免多层嵌套 `if`（Swift Book → 控制流章节：guard 语句）；③ 全部校验通过后赋值三个 `@Published` 属性并写入 `UserDefaults` 持久化。由于这三个属性都是 `@Published`，赋值的瞬间 `ContentView` 就会自动从 `LoginView` 切换到 `MainTabView`。
- 第 55~60 行 `func logout()`：与登录相反——把 `currentUser` 设为 `nil`、`isLoggedIn` 设为 `false`、清空错误信息，并把 `UserDefaults` 中的登录状态写为 `false`。同样因为 `@Published`，`ContentView` 会自动切回 `LoginView`，`ProfileView` 中点击「Log Out」最终就是调用这个方法。

知识点：`@MainActor`（并发隔离）/ `class` 引用类型与 `final` / `ObservableObject` + `@Published` (Combine) / `Optional` / `private` 访问控制 / `static` 类型属性 / 自定义 `init` / `guard` 提前返回 / 字符串方法 / 三元运算符 `?:` / `UserDefaults` 本地持久化

● Swift Book 参考章节：《并发》/ 《结构体与类》/ 《属性》/ 《基础语法》/ 《基本运算符》/ 《字符串与字符》/ 《控制流》/ 《访问控制》/ 《初始化》

5. Views/LoginView.swift — 登录界面

■■■■■ManhattanTravelApp/Views/LoginView.swift

登录页 UI：邮箱/密码输入框、密码显示/隐藏切换、错误提示文字、登录按钮。点击「Log In」会调用 `AuthManager.login(email:password:)`。

完整代码

```
import SwiftUI

struct LoginView: View {
    @EnvironmentObject var authManager: AuthManager
    @State private var email: String = ""
    @State private var password: String = ""
    @State private var isPasswordVisible: Bool = false

    var body: some View {
        ScrollView {
            VStack(spacing: 28) {
                header
                form
                Spacer(minLength: 0)
            }
            .padding(.bottom, 32)
        }
        .background(Color(.systemGroupedBackground))
    }

    private var header: some View {
        VStack(spacing: 12) {
            ZStack {
                Circle()
                    .fill(Color.blue.opacity(0.12))
                    .frame(width: 88, height: 88)
                Image(systemName: "map.fill")
                    .font(.system(size: 36))
                    .foregroundColor(.blue)
            }
            Text("OFFPEAK")
                .font(.system(size: 30, weight: .bold))
            Text("Sign in to plan your trip")
                .foregroundColor(.secondary)
        }
        .padding(.top, 60)
    }

    private var form: some View {
        VStack(alignment: .leading, spacing: 16) {
            labeledField(title: "Email") {
                TextField("you@example.com", text: $email)
                    .textInputAutocapitalization(.never)
                    .keyboardType(.emailAddress)
                    .autocorrectionDisabled()
            }

            labeledField(title: "Password") {
```

```

        HStack {
            Group {
                if isPasswordVisible {
                    TextField(".....", text: $password)
                } else {
                    SecureField(".....", text: $password)
                }
            }
            Button {
                isPasswordVisible.toggle()
            } label: {
                Image(systemName: isPasswordVisible ? "eye.slash" : "eye")
                    .foregroundColor(.gray)
            }
        }
    }

    if let errorMessage = authManager.errorMessage {
        Text(errorMessage)
            .font(.footnote)
            .foregroundColor(.red)
    }

    Button(action: login) {
        Text("Log In")
            .font(.headline)
            .foregroundColor(.white)
            .frame(maxWidth: .infinity)
            .padding()
            .background(Color.blue)
            .cornerRadius(14)
    }
    .padding(.top, 8)

    Text("Mock sign-in – any email and a 6+ character password works")
        .font(.caption)
        .foregroundColor(.secondary)
        .frame(maxWidth: .infinity, alignment: .center)
    }
    .padding(20)
    .background(Color(.systemBackground))
    .cornerRadius(20)
    .padding(.horizontal, 20)
}

private func login() {
    authManager.login(email: email, password: password)
}

```

```

@ViewBuilder
private func labeledField<Content: View>(
    title: String,
    @ViewBuilder content: () -> Content
) -> some View {
    VStack(alignment: .leading, spacing: 6) {
        Text(title)
            .font(.subheadline.bold())
        content()
            .padding()
            .background(Color(.systemGray6))
            .cornerRadius(12)
    }
}

#Preview {
    LoginView()
        .environmentObject(AuthManager())
}

```

代码讲解

- 第3行 `struct LoginView: View`：遵循 `View` 协议，必须实现计算属性 `var body: some View`。`some View` 表示「返回某个遵循 `View` 协议的具体类型，但调用者不需要关心具体是什么类型」——即「不透明返回类型 (opaque return type)」。

- 第4行 `@EnvironmentObject var authManager: AuthManager`：从环境中取出第1步在 App 里注入的同一个认证状态对象。（Swift Book → 属性章节：属性包装器）
- 第5~7行 `@State private var email / password / isPasswordVisible: @State` 是 SwiftUI 用来管理「视图私有、轻量」状态的属性包装器；值一旦改变，会自动触发本视图重新渲染。`private` 表示只在 `LoginView` 内部使用。（Swift Book → 属性章节：属性包装器；访问控制章节）
- `body` 用 `ScrollView { VStack { header; form; Spacer() } }`，把界面拆成 `header` 和 `form` 两个计算属性——这是 SwiftUI 中常见的「视图分解 (view decomposition)」写法，让 `body` 保持简洁、易读。（Swift Book → 属性章节：计算属性）
- `header` 里 `ZStack { Circle()...; Image(systemName: "map.fill")... }`：`ZStack` 让两个视图按 z 轴叠放，做出「圆形底 + 图标」的徽标效果；`Image(systemName:)` 直接使用系统自带的 SF Symbols 图标库。
- `form` 里 `TextField("you@example.com", text: $email): $email` 是 `@State` 属性包装器的「投影值 (projected value)」，类型是 `Binding<String>`——实现 UI 与 `email` 变量的「双向绑定」。（Swift Book → 属性章节：投影值 `projectedValue`）
- `.textInputAutocapitalization(.never)、.keyboardType(.emailAddress)、.autocorrectionDisabled()`：一连串「视图修饰符 (View Modifier)」，每个修饰符都返回一个「新的、被修饰过的视图」，链式调用，修饰符顺序有时影响最终效果。
- 密码框：`if isPasswordVisible { TextField(...) } else { SecureField(...) }` 根据 `@State` 在「明文」与「密文」之间切换；右侧切换按钮用了 `Button` 的「多尾随闭包 (multiple trailing closure)」写法——`Button { action ■■ } label: { ■■■■ }`。这是 Swift 闭包语法的重要特性：当函数的最后几个参数都是闭包时，可以依次写在括号外并用参数名标注 (`label:`)，让代码更像「声明式描述」而非函数调用。（Swift Book → 闭包章节：尾随闭包 `Trailing Closures`）
- `if let errorMessage = authManager.errorMessage { Text(errorMessage)... }`：「可选绑定 (optional binding)」，只有 `errorMessage` 不是 `nil` 时才会显示这段红色错误提示。（Swift Book → 基础语法章节：Optional Binding）
- `Button(action: logIn) { Text("Log In")... }`：把方法 `logIn` 作为「值」直接传给 `action:` 参数——函数在 Swift 中是「一等公民 (first-class citizen)」，可以像变量一样被传递，这正是闭包/函数值语法的体现。（Swift Book → 闭包章节：函数作为闭包）
- 文件末尾 `private func labeledField<Content: View>(title: String, @ViewBuilder content: () -> Content) -> some View`：① `<Content: View>` 是「泛型 (Generic)」写法——`Content` 是一个类型参数，约束必须遵循 `View` 协议，调用处会自动推断具体类型（Swift Book → 泛型章节）；② `content: () -> Content` 是一个「以闭包作为参数」的写法，调用时用尾随闭包语法写 UI（Swift Book → 闭包章节：将闭包作为参数传递）；③ `@ViewBuilder` 是一种「结果构建器 (Result Builder)」，赋予该参数支持在闭包体里写多个视图表达式的能力。

知识点：`View` 协议与 `body / some View` 不透明返回类型 / `@State` 与 `$` 投影出的 `Binding` / 视图分解（计算属性） / 视图修饰符链式调用 / `@ViewBuilder` 结果构建器 / `Button` 多尾随闭包 / 闭包作为参数 / 可选绑定 `if let` / 函数作为一等公民 / 泛型 `<T: Protocol>`

● Swift Book 参考章节：《闭包》 / 《属性》 / 《基础语法》 / 《泛型》 / 《函数》

6. Views/RegisterView.swift —— 注册页（占位骨架）

■■■■■ManhattanTravelApp/Views/RegisterView.swift

Xcode 新建 SwiftUI 文件时自动生成的最小模板，目前只显示一个「Register」文字，尚未实现注册表单与逻辑，是留给后续开发的占位骨架（TODO）。

完整代码

```
//  
// RegisterView.swift  
// ManhattanTravelApp  
//  
// Created by Sean on 15/06/2026.  
//  
import SwiftUI  
  
struct RegisterView: View {  
  
    var body: some View {  
        Text("Register")  
    }  
}
```

代码讲解

- 文件头部的注释块（文件名 / 所属模块 / 创建者 / 日期）是 Xcode 新建文件时自动写入的标准注释，不影响编译。
- `struct RegisterView: View { var body: some View { Text("Register") } }`：满足 View 协议的最简单写法——body 只要返回任意一个 View 即可，这里用最基础的 Text 视图占位，App 仍可正常编译运行。（Swift Book → 协议章节：最小协议实现）
- 后续可以参照 LoginView 的结构：补充 @State 表单字段（姓名 / 邮箱 / 密码 / 确认密码，对应后端 UserCreate 模型的字段）、注入 @EnvironmentObject var authManager: AuthManager，并在 AuthManager 中新增一个 register(...) 方法（同样走 mock 数据，注册成功后直接置 isLoggedIn = true）。

知识点：[View 协议的最小实现](#) / [Text 视图](#) / [代码占位骨架 \(TODO stub\)](#)

● Swift Book 参考章节：[《协议》](#) / [《结构体与类》](#)

7. Views/MainTabView.swift —— 主标签栏（登录后的主界面）

■■■■■ManhattanTravelApp/Views/MainTabView.swift

登录成功后展示的主界面：底部 5 个 Tab —— Explore / AI / Trip / Save / Profile，对应截图底部导航栏；其中 Profile 标签直接复用第 8 节的 ProfileView。

完整代码

```
import SwiftUI

struct MainTabView: View {
    var body: some View {
        TabView {
            PlaceholderTab(title: "Explore", systemImage: "safari")
                .tabItem {
                    Label("Explore", systemImage: "safari")
                }

            PlaceholderTab(title: "AI", systemImage: "sparkles")
                .tabItem {
                    Label("AI", systemImage: "sparkles")
                }

            PlaceholderTab(title: "Trip", systemImage: "calendar")
                .tabItem {
                    Label("Trip", systemImage: "calendar")
                }

            PlaceholderTab(title: "Save", systemImage: "bookmark")
                .tabItem {
                    Label("Save", systemImage: "bookmark")
                }

            ProfileView()
                .tabItem {
                    Label("Profile", systemImage: "person.crop.circle.fill")
                }
        }
    }
}

/// Placeholder for tabs outside the scope of the login/logout mockup.
private struct PlaceholderTab: View {
    let title: String
    let systemImage: String

    var body: some View {
        VStack(spacing: 12) {
            Image(systemName: systemImage)
                .font(.system(size: 48))
                .foregroundColor(.blue)
            Text(title)
                .font(.title2.bold())
        }
    }
}
```

```
#Preview {
    MainTabView()
        .environmentObject(AuthManager())
}
```

代码讲解

- `TabView { ... }`：SwiftUI 的容器视图，会自动在底部生成一个标签栏；`TabView` 内部的每个直接子视图就是一个标签页。
- `.tabItem { Label("Explore", systemImage: "safari") }`：用 `.tabItem` 修饰符为该标签配置图标和文字；`Label(title:systemImage:)` 是同时显示 SF Symbols 图标和文本的便捷视图。

- `private struct PlaceholderTab: View`: 把「大图标 + 大标题」这套重复的占位 UI 抽成一个可复用的私有视图, Explore / AI / Trip / Save 四个标签都复用它, 避免写四遍相同代码——体现「组件化 / DRY (Don't Repeat Yourself)」原则。`private` 限定该类型只在本文件内可见。(Swift Book → 结构体与类章节; 访问控制章节: `private`)
- 最后一个标签 `ProfileView()`: 直接复用第 8 节写好的个人主页视图, 对应截图里当前高亮的「Profile」标签和展示的页面内容。

知识点: `TabView` 与 `.tabItem/Label` (SF Symbols 图标 + 文字) / `private struct` 私有类型 / 组件化与视图复用

● Swift Book 参考章节: 《结构体与类》 / 《访问控制》

8. Views/ProfileView.swift — 一个人主页（含登出功能）★

■■■■■ManhattanTravelApp/Views/ProfileView.swift

完整还原截图中的 Profile 页：用户卡片、统计数字（Trips / Places / Off-peak）、设置列表（Notifications / Accessibility / Preferences / AI suggestions），并实现「Log Out」按钮与二次确认弹窗。

完整代码

```
import SwiftUI

struct ProfileView: View {
    @EnvironmentObject var authManager: AuthManager
    @State private var showLogoutConfirmation = false

    private var user: User {
        authManager.currentUser ?? AuthManager.mockUser
    }

    var body: some View {
        ScrollView {
            VStack(alignment: .leading, spacing: 20) {
                Text("Profile")
                    .font(.system(size: 36, weight: .bold))
                    .padding(.top, 8)

                profileCard
                statsRow
                settingsList
                logoutButton
            }
            .padding(.horizontal, 20)
            .padding(.bottom, 32)
        }
        .background(Color(.systemGroupedBackground))
        .confirmationDialog(
            "Are you sure you want to log out?",
            isPresented: $showLogoutConfirmation,
            titleVisibility: .visible
        ) {
            Button("Log Out", role: .destructive) {
                authManager.logout()
            }
            Button("Cancel", role: .cancel) {}
        }
    }

    private var profileCard: some View {
        HStack(spacing: 16) {
            ZStack {
                Circle()
                    .fill(Color(.systemGray5))
                    .frame(width: 64, height: 64)
                Image(systemName: "person.fill")
                    .font(.system(size: 28))
                    .foregroundColor(.gray)
            }
        }
    }
}
```

```

        VStack(alignment: .leading, spacing: 4) {
            Text(user.name)
                .font(.title3.bold())
            Text(user.joinedDate)
                .font(.subheadline)
                .foregroundColor(.secondary)
        }

        Spacer()

        if user.isPro {
            Text("Pro")
                .font(.subheadline.bold())
                .foregroundColor(.blue)
                .padding(.horizontal, 14)
                .padding(.vertical, 6)
                .background(Color.blue.opacity(0.15))
                .clipShape(Capsule())
        }
    }
    .padding(16)
    .background(Color(.systemBackground))
    .cornerRadius(20)
}

private var statsRow: some View {
    HStack(spacing: 12) {
        statCard(value: "\(user.tripsCount)", label: "Trips")
        statCard(value: "\(user.placesCount)", label: "Places")
        statCard(value: "\(user.offPeakPercentage)%", label: "Off-peak")
    }
}

private func statCard(value: String, label: String) -> some View {
    VStack(spacing: 6) {
        Text(value)
            .font(.system(size: 28, weight: .bold))
            .foregroundColor(.blue)
        Text(label)
            .font(.subheadline)
            .foregroundColor(.secondary)
    }
    .frame(maxWidth: .infinity)
    .padding(.vertical, 20)
    .background(Color(.systemBackground))
    .cornerRadius(16)
}

```

```

private var settingsList: some View {
    VStack(spacing: 0) {
        settingsRow(icon: "bell.fill", title: "Notifications", value: "On")
        Divider().padding(.leading, 60)
        settingsRow(icon: "figure.roll", title: "Accessibility", value: "Step-free")
        Divider().padding(.leading, 60)
        settingsRow(icon: "gearshape.fill", title: "Preferences", value: nil)
        Divider().padding(.leading, 60)
        settingsRow(icon: "sparkles", title: "AI suggestions", value: "Personalized")
    }
    .background(Color(.systemBackground))
    .cornerRadius(20)
}

private func settingsRow(icon: String, title: String, value: String?) -> some View {
    HStack {
        ZStack {
            RoundedRectangle(cornerRadius: 10)
                .fill(Color.blue.opacity(0.12))
                .frame(width: 36, height: 36)
            Image(systemName: icon)
                .foregroundColor(.blue)
        }
        Text(title)
            .font(.body.weight(.medium))
        Spacer()
        if let value {
            Text(value)
                .foregroundColor(.secondary)
        }
        Image(systemName: "chevron.right")
            .font(.footnote)
            .foregroundColor(Color(.systemGray3))
    }
    .padding(.vertical, 14)
    .padding(.horizontal, 16)
    .contentShape(Rectangle())
}

private var logoutButton: some View {
    Button {
        showLogoutConfirmation = true
    } label: {
        Text("Log Out")
            .font(.headline)
            .foregroundColor(.red)
            .frame(maxWidth: .infinity)

```

```

        .padding()
        .background(Color(.systemBackground))
        .cornerRadius(16)
    }
    .padding(.top, 8)
}

#Preview {
    ProfileView()
        .environmentObject(AuthManager())
}

```

代码讲解

- 第4~5行 `@EnvironmentObject var authManager: AuthManager` 与 `@State private var showLogoutConfirmation = false`：分别管理「全局共享的登录状态」和「本页面私有的弹窗显示状态」——两种状态管理方式的典型对比。
- 第7~9行 计算属性 `private var user: User { authManager.currentUser ?? AuthManager.mockUser } : ??` 是「nil 合并运算符 (nil-coalescing operator)」——如果左边的可选值是 nil，就用右边的默认值代替。（Swift Book → 基本运算符章节：nil 合并运算符）

- `body: ScrollView { VStack(alignment: .leading, spacing: 20) { Text("Profile")...; profileCard; statsRow; settingsList; logoutButton } }`——把整页拆成 4 个计算属性，结构清晰，一一对应截图里的四个区块。（Swift Book → 属性章节：计算属性）
- `.confirmationDialog("Are you sure...", isPresented: $showLogoutConfirmation, titleVisibility: .visible) { Button("Log Out", role: .destructive) { authManager.logout() }; Button("Cancel", role: .cancel) { } }`：系统原生的「操作表」弹窗，内容体本身也是一个 `@ViewBuilder` 闭包——SwiftUI 大量使用这种模式（Swift Book → 闭包章节）。`isPresented` 接收 `$showLogoutConfirmation (Binding<Bool>)`，值为 `true` 时弹出；`role: .destructive` 把按钮标红。
- `profileCard: ZStack { Circle().fill(...); Image(systemName: "person.fill")... }` 用「圆形 + 人形图标」做头像占位；`if user.isPro { Text("Pro")...clipShape(Capsule()) }` 仅当 `isPro == true` 时显示蓝色「Pro」胶囊徽章，`Capsule()` 是两端为半圆形的形状。
- `statsRow + private func statCard(value: String, label: String) -> some View: HStack(spacing: 12) { statCard(...); statCard(...); statCard(...) }` 水平排列三张统计卡片。`\(user.tripsCount)` 这种写法叫「字符串插值 (string interpolation)」，把变量的值嵌入到字符串里。（Swift Book → 字符串与字符章节：字符串插值）
- `settingsList + private func settingsRow(icon: String, title: String, value: String?) -> some View:` 用 `VStack(spacing: 0) { settingsRow(...); Divider(); ... }` 拼出带分割线的设置列表。`if let value { Text(value)... }` 是 Swift 5.7 引入的「简写可选绑定」，等价于 `if let value = value`，省去重复写变量名。（Swift Book → 基础语法章节：Optional Binding 简写形式）
- `logoutButton`：一个红色文字的按钮，点击后只做一件事——`showLogoutConfirmation = true`，从而弹出上面定义的确认对话框。这是「UI 触发 → 状态变化 → 系统自动展示弹窗」的典型 SwiftUI 声明式写法，全程没有手动调用任何 `present()` 方法。

知识点：`nil` 合并运算符 `??` / 视图分解为计算属性与私有方法 / `confirmationDialog` 与 `Binding` 驱动的弹窗（`@ViewBuilder` 闭包） / `Button` 的 `role` / `Capsule` 形状 / 简写可选绑定 `if let value` / 字符串插值 `\(...)` / `Divider` 与 `Spacer`

● Swift Book 参考章节：《基本运算符》 / 《闭包》 / 《属性》 / 《基础语法》 / 《字符串与字符》

9. 项目结构总览

本次新增的 iOS 工程位于仓库的 `ios/` 目录下，结构如下：

```
ios/
├── ManhattanTravelApp.xcodeproj/      Xcode
├── ManhattanTravelApp/                项目
│   ├── ManhattanTravelAppApp.swift    App
│   ├── ContentView.swift              主界面
│   ├── Models/                       数据模型
│   │   ├── User.swift                 用户
│   │   └── Managers/                 管理器
│   │       ├── AuthManager.swift      认证管理器
│   │       └── Views/                 视图
│   │           ├── LoginView.swift    登录页
│   │           ├── RegisterView.swift 注册页
│   │           ├── MainTabView.swift   主 TabView
│   │           ├── ProfileView.swift   个人资料页
│   │           └── Assets.xcassets/    资源
│   └── ...                             ...
└── ...                                 ...
```

数据流概览

- `ManhattanTravelAppApp` 创建唯一的 `AuthManager`，并通过 `.environmentObject(_)` 注入。
- `ContentView` 读取 `authManager.isLoggedIn`：为 `false` 时显示 `LoginView`，为 `true` 时显示 `MainTabView`（其中包含 `ProfileView`）。
- `LoginView` 中点击「Log In」→ 调用 `authManager.login(email:password:)` → 校验通过则 `isLoggedIn = true` → `ContentView` 自动切换到主界面。
- `ProfileView` 中点击「Log Out」→ 弹出确认框 → 确认后调用 `authManager.logout()` → `isLoggedIn = false` → `ContentView` 自动切回登录页。
- 整个过程依赖 `ObservableObject + @Published` 的「响应式」机制：状态变了，界面自动跟着变，不需要手写任何「刷新 UI」的代码。

闭包与尾随闭包语法	闭包是「可以作为值传递的代码块」，类似匿名函数。当函数的最后一个参数是闭包时，可以把它写在调用括号外的 <code>{ }</code> 中，即「尾随闭包 (trailing closure)」；若有多个闭包参数，除第一个外都要写参数名，如 <code>Button { █████ } label: { ███ }</code> 。 <code>@ViewBuilder</code> 赋予传入闭包体写多条视图表达式的能力。（Swift Book → 闭包章节：Trailing Closures 与 Multiple Trailing Closures）
字符串方法	<code>trimmingCharacters(in:)</code> 去除首尾指定字符（如空白/换行）； <code>contains("@")</code> 判断是否包含子串； <code>count</code> 返回字符数量； <code>"\(\████)"</code> 为字符串插值。均用于 <code>AuthManager.login</code> 的输入校验和 <code>ProfileView</code> 的数据展示。（Swift Book → 字符串与字符章节：String Interpolation 和字符串方法）
逐成员初始化器 (memberwise initializer)	未显式定义 <code>init</code> 的 <code>struct</code> ，编译器会自动生成一个按全部属性顺序传参的构造方法，如 <code>User(name:..., email:..., ...)</code> 。（Swift Book → 初始化章节：Memberwise Initializers for Structure Types）

B. SwiftUI 基础

View 协议与 body	SwiftUI 中所有界面元素都遵循 <code>View</code> 协议，唯一要求是实现 <code>var body: some View</code> ，返回这个视图「长什么样」。 <code>LoginView</code> 、 <code>ProfileView</code> 、 <code>MainTabView</code> 等都是如此。（Swift Book → 协议章节：Protocol Requirements）
some View （不透明返回类型）	表示「返回某个具体的、遵循 <code>View</code> 协议的类型，但调用者不需要、也不能知道具体是什么类型」。编译器在背后知道真实类型并做优化，开发者只需关心「这是一个 <code>View</code> 」。（Swift Book → 不透明类型章节）
@main / App 协议 / Scene / WindowGroup	<code>@main</code> 标记程序入口；遵循 <code>App</code> 协议的类型代表整个应用，需实现 <code>body: some Scene</code> ； <code>WindowGroup</code> 是最常见的 <code>Scene</code> ，代表一个窗口/界面容器。
#Preview 宏	Swift 5.9 引入，写在文件末尾，让 Xcode 的 Canvas 实时渲染该视图，便于边写边看效果，无需运行整个 App。

C. 状态管理与数据流

@State	用于视图「私有、轻量」的可变状态（如文本框内容、弹窗是否显示）。值改变会自动触发该视图重新渲染。声明时通常加 <code>private</code> 。（Swift Book → 属性章节：属性包装器 Property Wrappers）
@Binding 与 \$ 投影	在被 <code>@State</code> 等属性包装器修饰的变量前加 <code>\$</code> ，会得到一个 <code>Binding<T></code> ——指向原始数据的「引用」，可以传给子视图（如 <code>TextField</code> ）实现双向绑定：子视图改值，父视图的状态同步改变。（Swift Book → 属性章节：投影值 Projected Value）
@StateObject	用于创建并长期持有一个 <code>class</code> （遵循 <code>ObservableObject</code> ）的实例，保证它在视图生命周期内只被创建一次。 <code>ManhattanTravelApp</code> 用它创建 <code>authManager</code> 。（Swift Book → 属性章节：属性包装器）
@EnvironmentObject	从 SwiftUI 的「环境」中获取一个之前用 <code>.environmentObject(_:)</code> 注入的共享对象，无需手动逐层传递。本项目中 <code>authManager</code> 就是这样在 <code>ContentView</code> 、 <code>LoginView</code> 、 <code>ProfileView</code> 间共享的。（Swift Book → 属性章节：属性包装器）
ObservableObject + @Published (Combine)	<code>ObservableObject</code> 是一个标记协议；其中被 <code>@Published</code> 标记的属性一旦改变，会自动通过 <code>objectWillChange</code> 发布通知，所有订阅了该对象的视图都会刷新。 <code>AuthManager</code> 的 <code>isLoggedIn</code> 、 <code>currentUser</code> 、 <code>errorMessage</code> 均是 <code>@Published</code> 。（Swift Book → 协议章节；属性章节：属性包装器）

@MainActor	Swift 并发模型中的全局执行者标注，确保被标注类型的所有代码都在主线程（UI 线程）执行——更新界面相关状态时必须在主线程，否则会有运行时警告甚至崩溃。（Swift Book → 并发章节：Actors 与 Global Actors）
-------------------	--

D. 布局容器与常用控件

VStack / HStack / ZStack	三种最基础的布局容器：分别表示「垂直排列」「水平排列」「层叠（z 轴重叠）」子视图。
ScrollView	当内容可能超出屏幕高度时，包裹在 ScrollView 内即可让内容可滚动，LoginView 和 ProfileView 的最外层都是它。
TabView + .tabItem	TabView 自动生成底部标签栏，每个直接子视图是一个标签页；.tabItem 修饰符为该标签配置图标和文字。MainTabView 用它实现 Explore/Al/Trip/Save/Profile 五个标签。
Text / TextField / SecureField / Button / Image / Label	Text 显示静态文字；TextField 普通输入框；SecureField 密文输入框（用于密码）；Button 可点击按钮（支持多尾随闭包写法）；Image(systemName:) 显示 SF Symbols 图标；Label 同时显示图标和文字。（Swift Book → 闭包章节：多尾随闭包语法 Multiple Trailing Closures）
形状 Circle / RoundedRectangle / Capsule	Circle() 圆形、RoundedRectangle(cornerRadius:) 圆角矩形、Capsule() 两端半圆的胶囊形——常用于头像背景、图标背景、徽章背景。
Divider / Spacer	Divider() 画一条细分割线；Spacer() 在布局中占据「剩余空间」，常用来把内容推到一侧或撑满整行。
confirmationDialog	系统原生的底部操作表弹窗，通过 isPresented: Binding<Bool> 控制显示与隐藏，内部按钮可设置 role（如 .destructive 危险操作标红、.cancel 取消）。内容体是一个 @ViewBuilder 闭包，利用了 Swift 的多尾随闭包语法。（Swift Book → 闭包章节：@ViewBuilder / 多尾随闭包）

E. 视图修饰符 (View Modifiers)

.padding / .background / .cornerRadius / .frame	分别用于：增加内边距、设置背景（颜色/视图）、设置圆角、设置宽高/对齐等尺寸约束。这些修饰符可以链式调用，每次调用都返回一个「被包裹/修饰过的新视图」。
.font / .foregroundColor	设置文字字体/大小/字重，以及文字或图标的颜色，本项目大量使用 Color.blue、.secondary、Color(.systemGray6) 等系统色和语义色。
.animation(_:value:)	当 value 指定的值发生变化时，为相关的视图变化自动添加动画过渡。ContentView 用它让「登录页 ↔ 主界面」的切换更平滑。
修饰符顺序很重要	例如先 .padding() 再 .background(...) 会让背景包含内边距；反过来则背景只包住原始内容。本项目多处用 .padding(16).background(...).cornerRadius(20) 的固定顺序来实现「卡片」效果。

F. 数据持久化

UserDefaults	iOS 提供的轻量级键值存储，适合保存少量配置/状态（如「是否已登录」）。AuthManager 用 UserDefaults.standard.set(_:forKey:) 写入、bool(forKey:) 读取，实现「重启 App 后仍保持登录」。注意：它不适合存储密码等敏感信息（应使用 Keychain）。
---------------------	---